

Documento de Diseño y Análisis Técnico

Proyecto de Scratch: "Green Sheep vs. Nazi Sheep"

ID del Proyecto: 445947054 Plataforma: scratch.mit.edu Categoría: Videojuego / Carreras y Supervivencia

1. Resumen Ejecutivo del Proyecto

El proyecto bajo análisis corresponde a un videojuego interactivo bidimensional desarrollado en la plataforma educativa Scratch. La premisa central plantea un escenario de competencia asincrónica o de carrera de velocidad en la cual el usuario asume el rol del protagonista, denominado **"Green Sheep" (Oveja Verde)**. El objetivo principal consiste en sortear un entorno hostil repleto de obstáculos fijos o móviles para alcanzar una meta predeterminada (representada por una bandera) antes de que lo haga el antagonista automático, designado como **"Nazi Sheep"**.

Core Gameplay (Bucle Central): El jugador debe equilibrar la velocidad de desplazamiento para ganar la carrera contra la inteligencia artificial, manteniendo extrema precisión táctica para evitar la colisión con los obstáculos del entorno, bajo la penalización estricta de pérdida de una curva de vitalidad limitada a tres unidades.

2. Arquitectura de Juego y Mecánicas Principales

A nivel de diseño de sistemas de juego, el proyecto se estructura en base a tres pilares fundamentales que determinan las condiciones de victoria, derrota e interacción:

- **Mecánica de Carrera y Oposición Activa:** El jugador no solo compite contra el diseño del mapa, sino contra un temporizador implícito que es el avance del rival ("Nazi Sheep"). Esto añade un factor de urgencia psicológica, obligando al usuario a tomar riesgos y avanzar de forma continua en lugar de esperar pacientemente a que los obstáculos se retiren.
- **Sistema de Colisiones y Gestión de Salud:** El entorno está dotado de elementos punitivos (obstáculos). El motor de física simplificado de Scratch evalúa constantemente si el color o el borde de la "Green Sheep" intersecta con dichos objetos. Cada colisión efectiva activa un trigger de reducción en la variable global de vidas.
- **Condición de Fin de Partida (Game Over):** El sistema realiza una validación lógica con dos condicionales de salida. La victoria se alcanza si la coordenada de posición del jugador interseca con la bandera antes que el rival. La derrota ocurre si el rival llega primero o si la variable de vidas decrementa hasta llegar a cero de forma prematura.

3. Especificaciones Técnicas y Variables del Sistema

Para que un proyecto con estas características funcione correctamente bajo el entorno de ejecución de Scratch, se requiere la implementación y declaración de variables específicas. A continuación, se detalla la lógica de datos recomendada para la correcta ejecución del código subyacente:

Nombre de Variable	Tipo de Ámbito	Valor Inicial	Propósito / Función Mecánica
vidas_jugador	Global	3	Monitorea la salud actual del Green Sheep. Al llegar a 0, detiene el juego en estado de derrota.
velocidad_nazi_sheep	Local / Privada	Configurable	Determina la tasa de actualización en el eje X/Y para el avance del bot rival hacia la bandera.
estado_juego	Global	"Inicio"	Controla las pantallas de transiciones, tales como el menú principal, juego activo, victoria o derrota.

4. Propuesta de Pseudocódigo (Lógica de Bloques Scratch)

Dado que Scratch es un lenguaje de programación visual basado en bloques, la implementación del proyecto requiere de ciertos hilos de ejecución en paralelo (Al hacer clic en bandera verde). Se sugieren los siguientes patrones algorítmicos para los objetos principales:

A. Lógica del Jugador (Green Sheep)

El control se gestiona habitualmente mediante eventos del teclado (flechas de dirección o teclas WASD). El bloque crítico supervisa la salud y el entorno:

```
AL RECIBIR [Iniciar_Juego]
FIJAR [vidas_jugador] A (3)
IR A X:(-200) Y:(0) // Posición de salida
POR SIEMPRE
  SI ¿TOCANDO [Obstáculo]? ENTONCES
    CAMBIAR [vidas_jugador] POR (-1)
    IR A X:(-200) Y:(0) // Reinicio de posición por penalización
    ESPERAR (1) SEGUNDOS // Inmunidad temporal corta
  SI ¿TOCANDO [Bandera]? ENTONCES
    ENVIAR [Victoria_Jugador] Y PARAR TODO
  SI <(vidas_jugador) < (1)> ENTONCES
    ENVIAR [Game_Over] Y PARAR TODO
```

B. Lógica de la Inteligencia Artificial (Nazi Sheep)

El comportamiento del antagonista debe ser automatizado mediante un patrón de movimiento lineal o a través de puntos de ruta (waypoints) fijos que simulen un avance autónomo constante:

```
AL RECIBIR [Iniciar_Juego]
IR A X:(-200) Y:(-50) // Carril alterno o paralelo de salida
POR SIEMPRE
    DESLIZAR (8) SEG A X:(200) Y:(-50) // Movimiento progresivo a la meta
    SI ¿TOCANDO [Bandera]? ENTONCES
        ENVIAR [Derrota_Rival_Llego] Y PARAR TODO
```

5. Análisis de Interfaz, Estética y Experiencia de Usuario (UX)

Al desarrollar o modificar este proyecto, la claridad visual es esencial para garantizar una experiencia justa. La pantalla debe dividirse visualmente en tres áreas bien definidas:

- **Interfaz de Usuario de Datos (HUD):** Ubicada preferiblemente en la esquina superior izquierda, mostrando el contador de `vidas_jugador` utilizando variables visuales de Scratch en modo "grande" o mediante disfraces de corazones que se actualicen dinámicamente.
- **El Área de Juego (Layout del Nivel):** Un fondo contrastante que permita identificar sin ambigüedades los objetos peligrosos (rojo o negro) de los caminos seguros, facilitando la lectura de la trayectoria del "Green Sheep".
- **La Zona de Meta:** La bandera debe poseer un color llamativo y una animación constante para guiar el foco de atención del jugador desde el primer segundo de la partida.

6. Conclusiones y Recomendaciones de Optimización

El concepto de "Green Sheep vs. Nazi Sheep" presenta una estructura jugable clásica de "Carrera de Supervivencia", muy efectiva para el aprendizaje de condicionales lógicos en Scratch. Para llevar el proyecto a un nivel superior de calidad técnica, se sugieren las siguientes mejoras:

1. **Debounce en colisiones:** Asegurar el uso de un retardo tras tocar un obstáculo para evitar que múltiples cuadros (frames) de animación resten las 3 vidas de golpe en una sola colisión.
2. **Graduación de Dificultad:** Permitir que la velocidad del rival varíe según el rendimiento previo del jugador para evitar la frustración o el aburrimiento prematuro.